Conjuntos no Python

Lembrando que...

- Um conjunto (set) é uma coleção não ordenada de elementos únicos. Ele pode ser criado de duas maneiras: com a função set ou por meio de um conjunto literal com chaves:

```
>>> conjunto_processador = {'CPU', 'Registrador', 'Core'}
>>> conjunto_processador
    {'CPU', 'Registrador', 'Core'}
>>> type(conjunto_processador)
    <class 'set'>
```

```
>>> conj_proc = set(['CPU','Registrador','Core'])
>>> conj_proc
    {'CPU', 'Registrador', 'Core'}
>>> type(conj_proc)
    <class 'set'>
```

União

A máquina do usuário Thor está sendo monitorada a partir das medidas de 3 elementos de seu hardware. Mas a empresa resolveu ampliar o monitoramento de todas as máquinas, assim economizar no consumo de energia ou encontrar usuários subutilizando os computadores. A primeira medida foi a de capturar os dados de máquinas, e pelo menos dois processos em execução.

```
>>> user_thor = {'mysql','cpu','ram','ssd1','google'}
>>> user_thanos = {'lol','ram','cpu','hd','google'}
>>> user_ca = {'mysql','lol','ram','cpu','firefox'}
>>> user_ts = {'mysql','cpu','ram','ssd1','google'}
>>> inventario1 = user_thor | user_ca
>>> inventario1
{'firefox', 'ram', 'google', 'mysql', 'ssd1', 'cpu', 'lol'}
```

União

E se fosse realizada a operação de união com os 4 usuários, o resultado seria como o anterior? Confirme.

```
>>> user_thor = {'mysql','cpu','ram','ssd1','google'}
>>> user_thanos = {'lol','ram','cpu','hd','google'}
>>> user_ca = {'mysql','lol','ram','cpu','firefox'}
>>> user_ts = {'mysql','cpu','ram','ssd1','google'}
>>> inventario1 = user_thor | user_ca
>>> inventario1
{'firefox', 'ram', 'google', 'mysql', 'ssd1', 'cpu', 'lol'}
>>> inventario2 = user_thor | user_ca | user_thanos | user_ts
>>> inventario2
{'firefox', 'hd', 'ram', 'google', 'mysql', 'ssd1', 'cpu', 'lol'}
```

União

Podemos utilizar o comando `.union` também para realizar a união

```
>>> inventario3 = user_thor.union(user_thanos,user_ca,user_ts)
>>> inventario3
{'hd', 'firefox', 'ram', 'google', 'mysql', 'ssd1', 'cpu', 'lol'}
```

Intersecção

A resultante desta operação `&` ou `.intersection` de conjuntos distintos, com elementos finitos, é de se obter um novo conjunto com os elementos comuns em ambos conjuntos de origem.

```
>>> inventario4 = user_thor & user_ca
>>> inventario4
    {'mysql', 'cpu', 'ram'}
>>> inventario5 = user_thor.intersection(user_thanos)
>>> inventario5
    {'cpu', 'ram', 'google'}
```

Pertinência

Esta operação exibe como resultante a booleana, verdadeira ou falsa, para elemento pertence ao conjunto. Podendo-se utilizar o **`in`** ou o **`not in`**

```
>>> 'cpu' in user_ts
True
>>> 'firefox' in user_thanos
False
>>> 'lol' not in user_thor
True
>>> 'lol' not in user_thanos
False
```

Pertinência com subconjuntos e superconjuntos

Um caso bem comum é o de saber se determinado conjunto contém outro conjunto com todos os seus elementos, neste caso temos que um subconjunto está contido em um conjunto. Utilizando o `.issubset` ou `<=` e `.issuperset` ou `>=`

```
>>> user_thor.issubset(user_thanos)
False
>>> user_ts.issuperset(user_thor)
True
```

Igualdade

Vimos que conjuntos de dados, tem variedade e volume. Outra propriedade característica dos conjuntos é a **veracidade**.

A veracidade pode ser testada nos conjuntos de dados por sua igualdade, a comparação, até porque ela é dada pelos elementos que pertencem ao conjunto e não por sua classe, o conjunto em si.

```
>>> user_ts == user_thor
True
>>> user_ts != user_thor
False
```

Diferença simétrica

Relacionado a integridade de dados.

Um novo conjunto surge, contendo todos os elementos que não são comuns aos dois conjuntos.

```
>>> user_ts ^ user_ca
{'firefox', 'google', 'lol', 'ssd1'}
>>> user_ts.symmetric_difference(user_ca)
{'firefox', 'google', 'lol', 'ssd1'}
```

Adicionando ou removendo elementos

```
>>> user_thor.add('edge')
>>> user_thor
{'ram', 'google', 'edge', 'mysql', 'cpu', 'ssd1'}
>>>
>>> user_thor.remove('mysql')
>>> user_thor
{'ram', 'google', 'edge', 'cpu', 'ssd1'}
```

Compreensões de Conjuntos

Você define a compreensão de conjuntos entre chaves. Vamos criar um novo conjunto contendo apenas os valores pares exclusivos nos números da lista

```
>>> num = [1, 2, 2, 3, 4, 5, 6, 6, 7, 8, 9, 10, 10]
>>>
>>> par = { item for item in num if item % 2 == 0 }
>>> par
    {2, 4, 6, 8, 10}
>>> type(par)
    <class 'set'>
```

Principais métodos em Python para sets

Sintaxe	Sintaxe alternativa	Descrição
s.add()	n.a.	Adiciona um elemento ao set
s.clear()	n.a.	Remove todos os elementos de um set
s.copy()	n.a.	Retorna uma cópia do set
s.difference(t)	s – t	Retorna um set contendo as diferenças entre dois ou mais sets
s.difference_update(t)	s -= t	Remove os itens que estão no conjunto t do conjunto s
s.intersection_update(t)	s &= t	Faz com que o conjunto s contenha a interseção dele com t.
s.symetric_difference_update(t)	s ^= t	Faz com que o conjunto s contenha a diferença simétrica dele com t.
s.intersection(t)	s & t	Retorna um set, que é a interseção de dois sets

Principais métodos em Python para sets

Sintaxe	Sintaxe alternativa	Descrição
s.isdisjoin(t)	n.a.	Retorna True se s e t não tem nenhum item em comum
s.issubset(t)	<=	Retorna True se s é igual a ou um subconjunto de t.
s.issuperset(t)	>=	Retorna True se s é igual a ou t é um subconjunto de s.
s.discard(x)	n.a.	Remove o item x do conjunto s.
s.pop()	n.a.	Remove um elemento aleatório do set
s.remove(x)	n.a.	Remove o item x se ele estiver em s se não retorna um KeyError.
s.update(t)	s |= t	Adiciona cada item do conjunto s que não está no conjunto t para o conjunto t
s.union(t)	s | t	Todos os elementos únicos de s e t